Donweb-VPS-rodo — contexto completo del proyecto

> Documento de referencia para el proyecto **Donweb-VPS-rodo** en Claude. Generado desde el repositorio (las partes 2 y 3 se regeneran con `pnpm docs:pdf`). Vive en `docs/Donweb-VPS-rodo.md` (y en PDF en `docs/Donweb-VPS-rodo.pdf`) (último commit en `develop`: `4acde43 docs(deploy): real VPS data (DonWeb, port 5460, root bootstrap)`). Contiene: datos del proyecto y del servidor, estado actual, decisiones, la guía de despliegue paso a paso completa y copias exactas de los archivos de `deploy/`. **No contiene contraseñas ni claves**: esas viven solo en `/var/www/mora/shared/.env` del VPS y en el gestor de contraseñas.

Instrucciones sugeridas para el proyecto en Claude

Pegar en *Project instructions*:

```
Sos mi asistente para configurar y mantener el VPS de DonWeb donde se publica la web de Mora Machado Construcciones (Next.js 16 + PostgreSQL 16 + Nginx + PM2). Estoy aprendiendo: explicame cada paso, decime en qué máquina se ejecuta cada comando ([Mac] o [VPS]) y cómo comprobar que salió bien antes de seguir. Basate en el documento "Donweb-VPS-rodo" de este proyecto: la guía de despliegue y los archivos de deploy/ son la fuente de verdad. No inventes rutas, nombres ni comandos que contradigan ese documento. Nunca me pidas que pegue
contraseñas o claves en el chat. Respondé en español rioplatense.
```

---

1. Resumen del proyecto

Tema	Dato
Cliente	Mora Machado Construcciones — constructora familiar de Zárate (Bs. As.)
Qué es	Sitio público + panel `/admin` donde el cliente edita todo el contenido (textos, obras, servicios, blog, fotos, consultas).
Repositorio	`github.com/salvador-castro/mora-machado-construcciones` — hoy **público**, pasa a **privado** cuando esté en el VPS (el VPS clona con una *deploy key* de solo lectura).
Ramas	`develop` = web nueva (**ya está en GitHub**, push hecho el 16/10/2026). `main` = sitio viejo estático (HTML + 11 fotos) hasta el lanzamiento.

Tema	Dato
Stack	Next.js 16.4 (App Router, Cache Components, salida `standalone`) · React 19 · TypeScript · Tailwind v4 · Prisma 7.10 + PostgreSQL 16 · Node 22 · pnpm 12.11.2 (fijado en `packageManager`, se instala con Corepack).
Emails	**Resend** (solo envío) con plantillas React Email. Casillas info@, sofia@, rodo@ en **Webempresa**.
Hosting nuevo	VPS **DonWeb**, Ubuntu 24.04. Nginx (HTTPS con Let's Encrypt) → Next.js con PM2 en `127.0.0.1:3000` → PostgreSQL local.
Hosting actual	El sitio viejo sigue en **Vercel**. La migración al VPS se hace **solo con el OK del cliente**; antes, prueba en `nuevo.moramachadoconstrucciones.com`.

---

2. Datos del servidor

Dato	Valor
Proveedor	DonWeb (VPS)
Sistema	Ubuntu 24.04 (verificado: OpenSSH 9.6p1 Ubuntu) · plan 4 vCPU / 8 GB RAM / 30 GB NVMe / 2 TB
IP	`201.32.131.60` (IPv6 `2800:6c0:6::c04`, sin uso por ahora) · nombre `vps-6520179-x.dattaweb.com`
SSH	puerto **5460** (el 22 lo bloquea DonWeb). DonWeb entrega solo `root`; se crea `salva` (sudo) y se desactivan el ingreso de root y las contraseñas (guía, pasos 2.3 y 3.6). Atajo en la Mac: `ssh ssh-rodo` (llave `~/.ssh/rodo_vps`, `IdentitiesOnly yes`). Ojo: `vps-personal` es **otro** VPS de DonWeb (201.32.130.19, puerto 5086) — no confundir.
Zona horaria	`America/Argentina/Buenos_Aires` (los cron dependen de esto)
Firewall	`ufw`: 5460/tcp, 80/tcp, 443/tcp. DonWeb no filtra 80/443 (verificado 16/10). Postgres solo `localhost`.
App	PM2, proceso `mora`, **una sola instancia** fork, `node --env-file=/var/www/mora/shared/.env server.js`, `HOSTNAME=127.0.0.1`, `PORT=3000`.
Base	PostgreSQL 16, usuario `mora`, base `mora_prod`, `localhost:5432`.
Carpeta raíz	`/var/www/mora`

```
/var/www/mora/
├── repo/                clon del repositorio (historial completo; el seed lee fotos del commit 39b8b90)
├── releases/<fecha-sha>/ un git worktree por deploy (quedan las últimas 3)
├── current -> releases/… release activa (PM2 corre current/.next/standalone/server.js)
├── shared/.env          variables de producción (chmod 600)
├── uploads/             UPLOAD_DIR — fotos subidas desde el admin (Nginx las sirve directo). ⚠️ No se pueden perder.
├── backups/             db-AAAA-MM-DD.dump + uploads-AAAA-MM-DD.tar.gz (14 días)
└── logs/                app.out.log, app.err.log, backup.log, cleanup.log
/var/www/certbot/        webroot para los desafíos de Let's Encrypt
/etc/nginx/sites-available/  mora-staging-http, mora-staging (prueba) · mora-http, mora (dominio final)
```

---

3. Dominio, DNS y emails (estado verificado el 14/10/2026)

Cosa	Dónde
Registro del dominio `moramachadoconstrucciones.com`	**GoDaddy**
DNS	**Vercel** (nameservers `ns1.vercel-dns.com`, `ns2.vercel-dns.com`). Todos los registros se editan en Vercel → Domains → DNS Records.
Sitio viejo	Vercel (registros A del dominio y `www` apuntan a Vercel)
Casillas	**Webempresa** (info@, sofia@, rodo@)
MX	⚠️ **No existe ninguno**: hoy no llegan emails a esas casillas. Hay que cargar en Vercel los MX/SPF/DKIM que indique el cPanel de Webempresa (guía, paso 12). No depende del VPS.
TXT / DMARC	No hay ninguno todavía.

Registros a crear (resumen; valores exactos en los paneles):

Para qué	Name	Tipo	Valor
Prueba en el VPS	`nuevo`	A	IP del VPS
Casillas Webempresa	`@`	MX	servidor de Webempresa (+ prioridad)
Casillas Webempresa	`mail` (si corresponde)	A	IP de Webempresa
SPF Webempresa	`@`	TXT	`v=spf1 …` (del cPanel) — **único SPF de la raíz**

Para qué	Name	Tipo	Valor
DKIM Webempresa	`default._domainkey` (o el que indique)	TXT	`v=DKIM1; …`
DKIM Resend	`resend._domainkey`	TXT	`p=…` (del panel de Resend)
Rebotes Resend	`send`	MX	`feedback-smtp.<región>.amazonses.com` prioridad 10
SPF Resend	`send`	TXT	`v=spf1 include:amazonses.com ~all`
DMARC	`_dmarc`	TXT	`v=DMARC1; p=none; rua=mailto:info@moramachadoconstrucciones.com`
Lanzamiento	`@` y `www`	A	IP del VPS (quitando antes el dominio del **proyecto** de Vercel, nunca de la cuenta)

---

4. Datos de contacto confirmados por el cliente

- **WhatsApp principal y único número publicado:** 03487-656133 (= +54 9 3487 65-6133, `wa.me/5493487656133`). El 03487-514196 **no** se publica.
- Todo link de WhatsApp del sitio abre con un texto que dice que escriben **desde la web** (editable en Admin → Configuración → Teléfonos).
- **Email público y destinatario de las consultas:** info@moramachadoconstrucciones.com. Los avisos de consultas **no** llevan copia a nadie.
- Colores: se mantiene el acento actual por ahora.

---

5. Cómo funciona el deploy (resumen)

`/var/www/mora/current/deploy/deploy.sh <rama>` (la primera vez: `/var/www/mora/repo/deploy/deploy.sh develop`):

1. `git fetch` y nuevo **worktree** en `releases/<fecha>-<sha>` con `.env` → `shared/.env`.
2. `pnpm install --frozen-lockfile` (Corepack sin prompt: `COREPACK_ENABLE_DOWNLOAD_PROMPT=0`).
3. `pnpm prisma migrate deploy` (antes del build: el build lee la base).
4. `pnpm db:seed` — **idempotente**: en base vacía carga todo (textos, 11 fotos del sitio viejo, logo, favicon, admin no); en base con contenido solo agrega bloques/campos nuevos, nunca pisa ediciones.
5. `pnpm build` + copia de `public/` y `.next/static/` dentro de `.next/standalone/`.
6. `current` → release nueva y `pm2 startOrReload deploy/ecosystem.config.cjs`.
7. Chequeo `http://127.0.0.1:3000/healthz` (30 intentos × 2 s). Si falla → vuelve a la release anterior y sale con error.
8. `pm2 save` y borra releases viejas (quedan 3).

Después del primer deploy: `cd /var/www/mora/current && pnpm create-admin` y `crontab /var/www/mora/current/deploy/crontab`.

---

6. Reglas y trampas conocidas (importante)

1. **Nginx NO debe enviar `X-Forwarded-Proto`.** Con `https` en esa cabecera, Next 16 trata el rewrite a `/404` de `src/proxy.ts` como un fetch externo por HTTPS a sí mismo y responde **500** en slugs inexistentes. Las URLs absolutas salen de `SITE_URL`, la app no la necesita.
2. `X-Real-IP $remote_addr` **es obligatorio**: el rate limiting (login 5/15 min por cuenta+IP, formulario 5/15 min por IP, cambio de contraseña) usa esa cabecera.
3. **Una sola instancia de PM2** (fork). La caché de Next (`'use cache'` + `updateTag` desde el admin) vive en el proceso: dos instancias mostrarían contenido viejo. **Nada de** `proxy_cache` en Nginx.
4. `SITE_URL` **y** `SITE_NOINDEX` **se leen al buildear** → si cambian, hay que volver a correr `deploy.sh`. El resto de las variables se leen al arrancar (`pm2 reload … --update-env`).
5. `UPLOAD_DIR` **absoluto** (`/var/www/mora/uploads`): el server standalone corre desde otra carpeta.
6. **Migraciones aditivas**: se aplican mientras la versión anterior sigue sirviendo.
7. **404 reales**: `src/proxy.ts` consulta la DB para `/servicios|obras|blog/:slug` y reescribe a `/404` antes de renderizar (si no, con streaming sería un "soft 404" con status 200). Con Draft Mode (vista previa del admin) deja pasar.
8. **El seed necesita el historial de git** (commit `39b8b90`, fotos del sitio viejo) → los releases son worktrees, y el clon no puede ser `--depth 1`.
9. **Build y memoria**: en VPS de ≤2 GB de RAM, crear swap de 2 GB o el build muere con `Killed`.
10. **Primer** `git clone` **con deploy key** (`git@github.com:…`); si se clonó por https, `git remote set-url origin git@github.com:salvador-castro/mora-machado-construcciones.git`.
11. **Vercel**: al lanzar, quitar el dominio del **proyecto** viejo, **no** de la cuenta/equipo (se borraría la zona DNS con los registros de Webempresa y Resend).
12. **Un solo SPF en la raíz** (el de Webempresa). El de Resend va en `send`.
13. Las respuestas de `/uploads/` las sirve Nginx (no pasan por Next): por eso sus cabeceras están en `mora.conf`. CSP/HSTS/etc. las pone Next (`next.config.ts`); HSTS y `upgrade-insecure-requests` solo si `SITE_URL` es https.
14. Sin `RESEND_API_KEY` en producción, los avisos quedan como **FAILED** en la consulta (con botón «Reintentar envío»); la consulta se guarda igual.

---

7. Comandos útiles del proyecto (desde `/var/www/mora/current`)

Comando	Qué hace
`pnpm create-admin`	Crea/resetea un usuario ADMIN (interactivo).
`pnpm db:seed`	Contenido base idempotente (lo corre el deploy). `pnpm prisma db seed -- --force` **borra el contenido** (conserva usuarios y consultas): no usar en producción.

Comando	Qué hace
`pnpm db:cleanup`	Borra sesiones y rate limits vencidos (cron 03:45).
`pnpm db:check`	Verifica la conexión a Postgres.
`pnpm brand:import`	Sube logo/favicon a Medios y los asigna si están vacíos (lo hace el seed).
`deploy/backup.sh`	Backup manual (cron 03:15).
`curl http://127.0.0.1:3000/healthz`	`{"ok":true}` si app y base funcionan; 503 si la base no responde.

7.b Segundo proyecto en el mismo VPS (próximamente)

En 15–20 días empieza el **sistema de gestión** (empleados, registro diario, sueldos, obras, presupuestos con PDFs) en el **mismo VPS**. Plan acordado:

- Base y usuario de Postgres **propios** (datos sensibles: DNI, CUIL, sueldos), proceso PM2 propio en otro puerto (3001), carpeta propia (`/var/www/mora-gestion`) y subdominio propio (ej. `gestion.moramachadoconstrucciones.com`).
- Parametrizar `deploy.sh` , `ecosystem.config.cjs` , Nginx y backups (hoy asumen una sola app `mora` en el puerto 3000).
- No deployar las dos apps a la vez (dos builds simultáneos pueden quedarse sin RAM).
- Pasar el backup de fotos a incremental (hoy cada día es una copia completa).

---

8. Estado actual y pendientes

Hecho (código): sitio + admin completos (F1–F10), Resend, SEO/a11y (Lighthouse 100 en a11y/BP/SEO), seguridad (CSP, rate limits), 26 tests e2e verdes, CI de GitHub Actions, scripts de deploy probados en local (primer deploy con base vacía, backup y restore, Nginx con HTTPS local).

No probado todavía (se verifica en el VPS real):

- Reinicio y rollback automático con PM2 (en el entorno local PM2 no pudo hablar con su daemon).
- El workflow de CI (corre en el primer push).
- certbot y DNS reales.

Pasos pendientes, en orden:

1. ~~`git push -u origin develop`~~ ✅ hecho el 16/10/2026.
2. Configurar el VPS siguiendo la guía (pasos 2.2 a 11: llave SSH, crear `salva` desde root, seguridad, programas…) con `develop` en `nuevo.moramachadoconstrucciones.com` .
3. Cargar MX/SPF/DKIM de Webempresa en Vercel (paso 12) — urgente, hoy no llegan emails a las casillas.
4. Resend: dominio, DNS, API key (paso 13).
5. Backups a Google Drive con rclone (paso 14).

6. Monitoreo UptimeRobot sobre `/healthz` (paso 15).
7. OK del cliente → lanzamiento (paso 16): merge `develop` → `main`, `.env` a producción, DNS al VPS, certificado, `deploy.sh main`, repo a privado.

Pendientes de contenido del cliente (se ven en el inicio del admin):

- Elegir color de acento: lima (template) o terracota (marca actual).
- Fotos adicionales en alta resolución (las actuales son de 1200 px), logos de clientes y testimonios reales.
- Revisar textos redactados: intro de inicio, Nosotros, detalle de servicios, blog y política de privacidad.
- Confirmar rangos de presupuesto del formulario.
- Mejorar los textos alternativos de las fotos de galería (hoy «Detalle de obra»).

PARTE 2 — Guía de despliegue paso a paso (copia de `deploy/README.md`)

Guía de despliegue en el VPS

Esta guía está pensada para alguien que **hace su primer deploy en un VPS**. Explica qué hace cada paso, por qué, y cómo comprobar que salió bien antes de seguir. Seguila en orden y no saltees los "✅ Comprobá".

> **Situación hoy (octubre 2026):** el sitio **viejo** está publicado en **Vercel**. La web nueva se pasa al VPS **solo cuando el cliente dé el OK**. Por eso el plan tiene dos etapas:
>
> 1. **Prueba (staging):** se instala todo en el VPS y se publica en `https://nuevo.moramachadoconstrucciones.com`. El sitio viejo sigue igual.
> 2. **Lanzamiento:** con el OK del cliente, el dominio principal pasa a apuntar al VPS.

Índice

- 0. Conceptos rápidos
- 1. Qué hay hoy y qué necesitás antes de empezar
- 2. En tu Mac: subir el código y preparar la llave SSH
- 3. Primer ingreso al VPS y seguridad básica
- 4. Instalar los programas
- 5. Base de datos
- 6. Carpetas y variables de entorno

- 7. Acceso a GitHub (repo privado)
- 8. Primer deploy
- 9. DNS: el subdominio de prueba
- 10. Nginx y HTTPS para la prueba
- 11. Revisar la prueba
- 12. Emails: casillas en Webempresa
- 13. Emails del sitio: Resend
- 14. Backups a Google Drive
- 15. Monitoreo
- 16. Lanzamiento (con el OK del cliente)
- 17. El día a día
- 18. Si algo falla
- Anexo: cómo funciona por dentro

---

0. Conceptos rápidos

Término	Qué es
VPS	Una computadora Linux en la nube (DonWeb) que es toda tuya. No tiene pantalla: la manejás por comandos.
SSH	La forma de conectarte al VPS desde tu terminal. Todo lo que escribas se ejecuta en el VPS.
Llave SSH	Un par de archivos (privado + público) que reemplaza a la contraseña. Más seguro, y no tenés que tipear nada.
DNS	La "agenda" de internet: dice que `moramachadoconstrucciones.com` está en tal IP. Cambiar el DNS es lo que "muda" el sitio.
Registro A	Entrada del DNS: "este nombre → esta IP".
Registro MX	Entrada del DNS: "los emails de este dominio van a tal servidor".
Registro TXT	Texto en el DNS. Se usa para demostrar que el dominio es tuyo y para que los emails no vayan a spam (SPF, DKIM, DMARC).
Nginx	El "portero" del VPS. Recibe las visitas (puertos 80 y 443), maneja el HTTPS, sirve las fotos y le pasa el resto a la app.
PM2	Mantiene la app de Next.js corriendo: la arranca, la reinicia si se cae y la levanta sola si el VPS se reinicia.
Let's Encrypt / certbot	Certificados HTTPS gratis (el candadito). `certbot` los pide y los renueva solo cada ~60 días.
`.env`	Archivo con las claves y la configuración de producción. **Nunca** se sube a GitHub.
Release	Cada deploy arma una copia nueva y completa de la app en una carpeta propia. Si algo sale mal, se vuelve a la anterior.

Cómo leer los comandos de esta guía:

- **[Mac]** = se ejecuta en la terminal de tu Mac.
- **[VPS]** = se ejecuta conectado al VPS por SSH.
- Lo que está entre `<así>` lo reemplazás vos (sin los `<>`). Ej.: `<CONTRASEÑA_DB>` → la contraseña que generaste.
- Copiá los comandos **de a uno** y mirá qué responde antes de seguir.

---

1. Qué hay hoy y qué necesitás antes de empezar

Cómo está el dominio hoy (verificado el 14/10/2026)

Cosa	Dónde está
Registro del dominio	**GoDaddy** (ahí se paga la renovación anual).
DNS (los registros)	**Vercel**: los nameservers son `ns1.vercel-dns.com` y `ns2.vercel-dns.com`. Los registros se editan en el panel de Vercel → **Domains**.
Sitio web viejo	Vercel.
Casillas info@, sofia@, rodo@	**Webempresa**.
Registros MX	⚠️ **No hay ninguno.** Con el DNS así, a esas casillas **no les llegan emails**. Se arregla en el paso 12.

> Podés verlo vos mismo en cualquier momento **[Mac]**:
>
> ```
> dig +short NS moramachadoconstrucciones.com     # quién maneja el DNS
> dig +short A  moramachadoconstrucciones.com     # a qué IP apunta el sitio
> dig +short MX moramachadoconstrucciones.com     # a dónde van los emails (hoy: vacío)
> ```

Datos del VPS (DonWeb)

Dato	Valor
IP (IPv4)	`201.32.131.60`
IPv6	`2800:6c0:6::c04` (no la usamos por ahora: no se crean registros `AAAA`)
Nombre del servidor	`vps-6520179-x.dattaweb.com`
Sistema	Ubuntu 24.04 (verificado: OpenSSH 9.6p1 Ubuntu)
SSH	puerto **5460** (el 22 está bloqueado por DonWeb)

Dato	Valor
Usuario inicial	`root` (contraseña en el panel de DonWeb). En el paso 2.3 creamos el usuario `salva` y después se desactiva el ingreso de `root`.
Puertos 80 y 443	Llegan al VPS sin bloqueo de DonWeb (verificado): no hace falta abrir nada en el panel.

Checklist: tené esto a mano

- [ ] Panel de **DonWeb**: la contraseña de `root` (la IP y el puerto están en la tabla de arriba).
- [ ] Acceso al **panel de Vercel** de la cuenta donde está el dominio (para editar el DNS).
- [ ] Acceso al **cPanel de Webempresa** (para los registros de las casillas).
- [ ] Tu cuenta de **GitHub** con permisos de administrador del repo.
- [ ] Una cuenta de **Resend** (gratis, resend.com).
- [ ] Tu cuenta de **Google** (para los backups en Drive).
- [ ] Un gestor de contraseñas o una nota segura para guardar las claves que vas a generar.

---

2. En tu Mac: subir el código y preparar la llave SSH

2.1 Subir el código a GitHub

El VPS descarga el código desde GitHub, así que primero tiene que estar ahí. **[Mac]**, en la carpeta del proyecto:

```
git status                # no debería haber cambios sin commitear
git push -u origin develop
```

✅ **Comprobá:** en GitHub, en el selector de ramas, aparece `develop` con el último commit.

> Mientras se prueba, el VPS usa la rama `develop`. En el lanzamiento se mergea a `main` (paso 16).

2.2 Crear una llave SSH solo para este servidor

Usamos una llave **exclusiva** para el VPS de Mora Machado (`~/.ssh/rodo_vps`), igual que `vps-personal` usa `~/.ssh/donweb_vps`. Así, si algún día hay que revocar una, no afecta a los otros servidores.

[Mac]:

```
ls ~/.ssh/rodo_vps.pub
```

- Si dice "No such file", creala (podés poner una frase de seguridad o dejarla vacía con Enter):

```
ssh-keygen -t ed25519 -C "salva-mac rodo" -f ~/.ssh/rodo_vps
```

- Si ya existe, seguí.

2.3 Primer ingreso como `root` y creación de tu usuario

DonWeb entrega el VPS con el usuario `root` (el administrador total). Trabajar siempre como `root` es peligroso: un error de tipeo puede romper el sistema, y es el usuario que todos los bots intentan adivinar. Por eso creamos un usuario propio, `salva`, que puede hacer tareas de administrador anteponiendo `sudo`, y después cerramos el ingreso de `root`.

[Mac] — conectate como root (te pide la contraseña del panel de DonWeb):

```
ssh -o PubkeyAuthentication=no -p 5460 root@201.32.131.60
```

> `-o PubkeyAuthentication=no` hace que pida directamente la contraseña: como tenés varias llaves en la Mac, sin esto SSH las prueba todas y el servidor corta con `Too many authentication failures`.

- La primera vez pregunta `Are you sure you want to continue connecting (yes/no)?`: escribí `yes`.
- Al tipear la contraseña **no se ve nada** en pantalla (ni asteriscos): es normal. Pegala con `Cmd+V` y Enter.
- Si te pide cambiar la contraseña de root, elegí una nueva y guardala en tu gestor.

[VPS, como root] — creá el usuario y dale permisos de administrador:

```
adduser salva
```

Te pide una contraseña para `salva` (guardala en tu gestor: la vas a usar con `sudo`) y datos opcionales (nombre, teléfono…): Enter en todos y `Y` al final.

```
usermod -aG sudo salva          # lo suma al grupo de administradores
id salva                        # tiene que aparecer "sudo" en la lista de grupos
exit                            # salís del VPS
```

2.4 Copiar tu llave al usuario `salva`

[Mac] (te pide la contraseña de `salva` **una última vez**):

```
ssh-copy-id -i ~/.ssh/rodo_vps.pub -o IdentitiesOnly=yes -p 5460 salva@201.32.131.60
```

2.5 Atajo para conectarte

[Mac]: abrí (o creá) `~/.ssh/config`:

```
nano ~/.ssh/config
```

Agregá al final (debajo de `vps-personal`, sin tocar los otros bloques):

```
Host ssh-rodo
    HostName 201.32.131.60
    Port 5460
    User salva
    IdentityFile ~/.ssh/rodo_vps
    IdentitiesOnly yes
    AddKeysToAgent yes
    UseKeychain yes
```

Guardá con `Ctrl+O`, Enter, y salí con `Ctrl+X`.

> **Por qué `IdentitiesOnly yes`:** tenés varias llaves (`donweb_vps`, `id_ed25519_lcaba`, …). Sin esa línea, SSH las prueba todas y el servidor puede cortar con `Too many authentication failures` antes de llegar a la correcta.

✅ **Comprobá:**

```
ssh ssh-rodo            # entra SIN pedir contraseña
sudo -v                 # (ya en el VPS) pide la contraseña de salva y no da error
exit
```

> De acá en adelante **siempre** entrás con `ssh ssh-rodo` (usuario `salva`), nunca como root.

---

3. Primer ingreso al VPS y seguridad básica

Conectate: **[Mac]** `ssh ssh-rodo`. Desde acá, todo es **[VPS]** hasta que se diga lo contrario.

3.1 Ver qué hay instalado

Antes de instalar nada, mirá si el VPS ya tiene cosas (por ejemplo, otro sitio):

```
lsb_release -a            # debería decir Ubuntu 24.04
sudo ss -tlnp             # qué programas están escuchando en qué puertos
ls /etc/nginx/sites-enabled/ 2>/dev/null
```

> Si ves algo en los puertos 80, 443, 3000 o 5432, o hay sitios en `/etc/nginx/sites-enabled/` que no sean `default`, **frená y averiguá qué es** antes de seguir. Esta guía asume un VPS dedicado al sitio.

3.2 Actualizar el sistema y la hora

```
sudo apt update && sudo apt upgrade -y
sudo timedatectl set-timezone America/Argentina/Buenos_Aires
timedatectl               # tiene que decir "America/Argentina/Buenos_Aires"
```

La zona horaria importa porque los backups se programan "a las 03:15" de la hora del servidor.

3.3 Firewall

El firewall (`ufw`) bloquea todo lo que no autoricemos.

> ⚠️ **Muy importante:** autorizá el puerto SSH (**5460**) **antes** de activar el firewall. Si no, te quedás afuera del VPS y vas a tener que entrar por la consola del panel de DonWeb.

```
sudo apt install -y ufw
sudo ufw allow 5460/tcp          # SSH (puerto de DonWeb)
sudo ufw allow 80/tcp            # http (para certbot y redirigir a https)
sudo ufw allow 443/tcp           # https
sudo ufw enable                  # responde "y"
sudo ufw status                  # tienen que aparecer 5460, 80 y 443 como ALLOW
```

> DonWeb no bloquea 80 ni 443 en este VPS (verificado). Si algún día agregan un firewall en el panel, hay que abrir ahí los mismos puertos.

✅ **Comprobá:** abrí **otra** terminal en tu Mac y hacé `ssh ssh-rodo`. Si entra, el firewall está bien. Recién ahí cerrá la sesión vieja.

3.4 Memoria de intercambio (swap)

El build de la app usa bastante memoria. Si el VPS tiene poca RAM (2 GB o menos), el build puede cortarse con un error tipo `Killed`. Un archivo de swap de 2 GB lo evita:

```
free -h                                  # mirá la fila "Swap"
# Si Swap dice 0B:
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
free -h                                  # ahora Swap debería decir 2.0Gi
```

3.5 Actualizaciones de seguridad automáticas

```
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades   # elegí "Yes"
```

3.6 Cerrar el ingreso de `root` y las contraseñas por SSH

Solo después de comprobar que entrás con la llave como `salva` (paso 2.5). Así nadie puede adivinar contraseñas ni entrar como `root`.

```
sudo tee /etc/ssh/sshd_config.d/01-hardening.conf > /dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
EOF
sudo sshd -t && sudo systemctl reload ssh
```

> **Por qué el archivo se llama `01-…`:** SSH lee los archivos de `/etc/ssh/sshd_config.d/` en orden alfabético y se queda con el **primer** valor que encuentra. Ubuntu suele traer `50-cloud-init.conf` con `PasswordAuthentication yes`; si nuestro archivo se llamara `90-…`, ese "yes" ganaría.

✅ **Comprobá:**

```
sudo sshd -T | grep -E "^(permitrootlogin|passwordauthentication)"
# tiene que decir:  permitrootlogin no   y   passwordauthentication no
```

y, **sin cerrar la sesión actual**, en otra terminal de tu Mac:

```
ssh ssh-rodo                                # sigue entrando (con la llave)
ssh -o PubkeyAuthentication=no -p 5460 root@201.32.131.60   # tiene que decir "Permission denied"
```

> Si te quedaras afuera: panel de DonWeb → **consola** del VPS (entrás como root desde el navegador) y borrás `/etc/ssh/sshd_config.d/01-hardening.conf`.

---

4. Instalar los programas

[VPS]

4.1 Paquetes base

```
sudo apt install -y git curl build-essential nginx certbot python3-certbot-nginx rclone \
  postgresql postgresql-contrib
```

4.2 Node.js 22

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs
node -v          # tiene que decir v22.algo
```

4.3 pnpm (con Corepack) y PM2

El proyecto fija la versión de pnpm en `package.json`. Corepack, que viene con Node, descarga esa versión exacta.

```
sudo corepack enable
echo 'export COREPACK_ENABLE_DOWNLOAD_PROMPT=0' >> ~/.bashrc && source ~/.bashrc
sudo npm install -g pm2
pm2 -v
```

Configurá PM2 para que arranque solo si el VPS se reinicia:

```
pm2 startup systemd
```

Ese comando **imprime otro comando** que empieza con `sudo env PATH=...`. Copialo entero y ejecutalo.

Rotación de logs (para que no llenen el disco):

```
pm2 install pm2-logrotate
pm2 set pm2-logrotate:max_size 10M
pm2 set pm2-logrotate:retain 30
```

✅ **Comprobá:** `node -v`, `pm2 -v`, `nginx -v`, `psql --version` (tiene que decir **16**) y `rclone version` responden sin error.

> Si `psql --version` no dice 16, avisá antes de seguir: los backups necesitan que coincida la versión.

---

5. Base de datos

PostgreSQL ya quedó instalado y corriendo. Ahora creamos un usuario y una base solo para el sitio.

5.1 Generar una contraseña para la base

```
openssl rand -hex 24
```

Copiá el resultado (son letras y números, sin caracteres raros, a propósito) y **guardalo en tu gestor de contraseñas** como "Postgres mora (VPS)".

5.2 Crear el usuario y la base

```
sudo -u postgres psql -c "CREATE USER mora WITH PASSWORD '<CONTRASEÑA_DB>';"
sudo -u postgres createdb -O mora mora_prod
```

✅ **Comprobá:**

```
psql "postgresql://mora:<CONTRASEÑA_DB>@localhost:5432/mora_prod" -c "select 1;"
```

Tiene que responder una tablita con un `1`.

> La base solo acepta conexiones desde el propio VPS (`localhost`): no es accesible desde internet, y está bien que así sea.

---

6. Carpetas y variables de entorno

6.1 Carpetas

```
sudo mkdir -p /var/www/mora/{releases,shared,uploads,backups,logs} /var/www/certbot
sudo chown -R salva:salva /var/www/mora
chmod 755 /var/www/mora /var/www/mora/uploads
```

Qué es cada una:

```
/var/www/mora/
├── repo/        copia del repositorio (se crea en el paso 7)
├── releases/    una carpeta por cada deploy (quedan las últimas 3)
├── current      "atajo" que apunta a la release activa
├── shared/.env  configuración de producción (una sola, la usan todas las releases)
├── uploads/     fotos que sube el cliente desde el admin ⚠️ no se pueden perder
├── backups/     copias diarias
└── logs/        registros de la app, backups y limpieza
```

6.2 Generar las claves secretas

```
openssl rand -base64 32     # → AUTH_SECRET  (firma las sesiones del admin)
openssl rand -base64 16     # → IP_HASH_SALT (anonimiza las IPs de las consultas)
```

Guardá ambos resultados.

6.3 Crear el archivo `.env`

```
nano /var/www/mora/shared/.env
```

Pegá esto **completando los valores**. Esta es la configuración para la etapa de **prueba** (staging):

```
# Base de datos (paso 5)
DATABASE_URL="postgresql://mora:<CONTRASEÑA_DB>@localhost:5432/mora_prod?schema=public"

# Sitio — PRUEBA en el subdominio (en el lanzamiento se cambia, ver paso 16)
SITE_URL="https://nuevo.moramachadoconstrucciones.com"
SITE_NOINDEX="true"

# Fotos subidas
UPLOAD_DIR="/var/www/mora/uploads"

# Seguridad (paso 6.2)
AUTH_SECRET="<resultado de openssl rand -base64 32>"
IP_HASH_SALT="<resultado de openssl rand -base64 16>"

# Emails del sitio (paso 13). Vacía = no se envían y quedan como fallidos en el admin.
RESEND_API_KEY=""
MAIL_FROM="Web Mora Machado <no-reply@moramachadoconstrucciones.com>"
MAIL_TO_DEFAULT="info@moramachadoconstrucciones.com"
LEAD_CONFIRMATION_EMAIL="false"

# Anti-spam opcional de Cloudflare (dejar vacío)
TURNSTILE_SITE_KEY=""
TURNSTILE_SECRET_KEY=""

# Backups (paso 14)
BACKUP_KEEP_DAYS="14"
BACKUP_RCLONE_REMOTE=""
BACKUP_REMOTE_KEEP_DAYS="90"
```

Guardá (`Ctrl+O` , Enter, `Ctrl+X`) y protegé el archivo para que solo lo pueda leer tu usuario:

```
chmod 600 /var/www/mora/shared/.env
```

Qué hace cada variable:

Variable	Para qué sirve
`DATABASE_URL`	Dónde está la base y con qué usuario entrar.
`SITE_URL`	La dirección pública del sitio, sin `/` al final. Se usa en el sitemap, en los links de los emails y en las vistas previas en redes. **Se lee al buildear**: si la cambiás, hay que volver a deployar.
`SITE_NOINDEX`	`true` = le pide a Google que **no** indexe el sitio. Para la prueba, así no compite con el sitio real. También se lee al buildear.

Variable	Para qué sirve
`UPLOAD_DIR`	Carpeta de las fotos. Siempre una ruta **absoluta** (que empieza con `/`).
`AUTH_SECRET`	Firma las sesiones del admin. Si la cambiás, todos los usuarios tienen que volver a iniciar sesión.
`IP_HASH_SALT`	Se usa para no guardar las IPs reales de quienes consultan (privacidad).
`RESEND_API_KEY`	Llave para enviar emails (paso 13).
`MAIL_FROM`	Remitente de los emails del sitio.
`MAIL_TO_DEFAULT`	A quién avisar de las consultas si en el admin no hay emails cargados.
`LEAD_CONFIRMATION_EMAIL`	`true` = además manda un "recibimos tu consulta" a quien escribió.
`BACKUP_*`	Retención de backups locales y en Google Drive (paso 14).

> 🔒 Nunca copies este archivo a GitHub, a un chat ni a un email. Si una clave se filtra, generá otra y reemplazala.

---

7. Acceso a GitHub (repo privado)

Cuando el repo sea **privado**, el VPS necesita permiso para descargarlo. Se usa una **deploy key**: una llave SSH que **solo puede leer este repo** (no tu cuenta entera).

> Si al momento de hacer esto el repo todavía es público, igual configurala: así no se rompe nada el día que lo pases a privado.

7.1 Crear la llave en el VPS

[VPS]

```
ssh-keygen -t ed25519 -C "vps-mora-deploy" -f ~/.ssh/github_mora -N ""
cat ~/.ssh/github_mora.pub
```

Copiá la línea que imprime (empieza con `ssh-ed25519`).

7.2 Cargarla en GitHub

En el navegador: repo `mora-machado-construcciones` → **Settings** → **Deploy keys** → **Add deploy key**.

- **Title:** `VPS DonWeb`
- **Key:** pegá la línea.
- **Allow write access:** ❌ **sin tildar** (solo lectura).

7.3 Decirle al VPS que use esa llave para GitHub

[VPS]

```
cat >> ~/.ssh/config <<'EOF'
Host github.com
    IdentityFile ~/.ssh/github_mora
    IdentitiesOnly yes
EOF
chmod 600 ~/.ssh/config
ssh -T git@github.com
```

La primera vez pregunta si confiás en el servidor: escribí `yes`.

✅ **Comprobá:** responde algo como `Hi salvador-castro/mora-machado-construcciones! You've successfully authenticated...`.

7.4 Clonar el repo

```
git clone git@github.com:salvador-castro/mora-machado-construcciones.git
/var/www/mora/repo
```

> Si alguna vez lo habías clonado con `https://`, cambiá la dirección con: `git -C /var/www/mora/repo remote set-url origin git@github.com:salvador-castro/mora-machado-construcciones.git`

---

8. Primer deploy

El script `deploy/deploy.sh` hace todo el trabajo. Para la prueba usamos la rama `develop`:

[VPS]

```
/var/www/mora/repo/deploy/deploy.sh develop
```

Tarda **entre 5 y 15 minutos** la primera vez. Vas a ver estas etapas:

Etapa	Qué hace
`► Actualizando repo`	Baja la última versión de GitHub y arma una carpeta nueva en `releases/`.

Etapa	Qué hace
`► Instalando dependencias`	Descarga las librerías (la primera vez es lo más lento).
`► Migraciones`	Crea o actualiza las tablas de la base.
`► Contenido base`	**La primera vez** carga los textos, las 11 fotos del sitio viejo, el logo y el ícono. En los deploys siguientes solo agrega lo nuevo y **nunca pisa lo que editó el cliente**.
`► Build`	Arma la versión optimizada del sitio.
`► Activando release`	Apunta `current` a la release nueva y (re)arranca la app con PM2.
`► Chequeo de salud`	Comprueba que la app responda. **Si no responde, vuelve sola a la versión anterior.**

Termina con `✓ Deploy <código> listo.`

✅ **Comprobá:**

```
pm2 status                              # "mora" en estado "online"
curl http://127.0.0.1:3000/healthz      # {"ok":true}
```

8.1 Crear el usuario administrador del cliente

```
cd /var/www/mora/current
pnpm create-admin
```

Te pide email, nombre y contraseña (mínimo 10 caracteres). Creá:

- uno para vos, y
- uno para el cliente (o creáselo después desde el admin, en **Usuarios**).

8.2 Programar backups y limpieza automáticos

```
crontab -l                                      # si dice "no crontab for salva", está
vacío
crontab /var/www/mora/current/deploy/crontab    # ⚠️ reemplaza el crontab actual
crontab -l                                      # tienen que aparecer las 2 tareas
```

> Si `crontab -l` ya tenía tareas, no uses el comando de arriba: abrí `crontab -e` y pegá a mano el contenido de `deploy/crontab` al final.

---

9. DNS: el subdominio de prueba

Vamos a crear `nuevo.moramachadoconstrucciones.com` apuntando al VPS. **No afecta al sitio actual ni a los emails.**

En el navegador: **Vercel** → (equipo donde está el dominio) → **Domains** → `moramachadoconstrucciones.com` → **DNS Records** → **Add Record**:

Campo	Valor
Name	`nuevo`
Type	`A`
Value	`201.32.131.60`
TTL	`60`

✅ **Comprobá [Mac]** (puede tardar unos minutos):

```
dig +short nuevo.moramachadoconstrucciones.com      # tiene que devolver 201.32.131.60
```

---

10. Nginx y HTTPS para la prueba

Los archivos de configuración están en `deploy/nginx/`, escritos para el dominio final. Para la prueba hacemos una copia reemplazando el dominio por `nuevo.…`.

[VPS]

10.1 Configuración del puerto 80 (http)

```
cd /var/www/mora/current/deploy/nginx
sed 's/moramachadoconstrucciones\.com/nuevo.moramachadoconstrucciones.com/g' mora-
http.conf \
  | sudo tee /etc/nginx/sites-available/mora-staging-http > /dev/null
sudo ln -s /etc/nginx/sites-available/mora-staging-http /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t                       # tiene que decir "syntax is ok" y "test is
successful"
sudo systemctl reload nginx
```

10.2 Pedir el certificado HTTPS

```
sudo certbot certonly --webroot -w /var/www/certbot \
  -d nuevo.moramachadoconstrucciones.com \
  --deploy-hook "systemctl reload nginx"
```

La primera vez te pide un email (para avisos de vencimiento) y aceptar los términos.

✅ **Comprobá:** termina con `Successfully received certificate`.

> Si falla con un error de "challenge", casi siempre es porque el DNS del paso 9 todavía no propagó o el puerto 80 está cerrado (paso 3.3).

10.3 Configuración https

```
cd /var/www/mora/current/deploy/nginx
sed 's/moramachadoconstrucciones\.com/nuevo.moramachadoconstrucciones.com/g' mora.conf \
  | sudo tee /etc/nginx/sites-available/mora-staging > /dev/null
sudo ln -s /etc/nginx/sites-available/mora-staging /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

> La copia incluye un bloque para `www.nuevo.…` que nunca recibe visitas: es inofensivo.

✅ **Comprobá:** abrí `https://nuevo.moramachadoconstrucciones.com` en el navegador. Tiene que cargar la web nueva con el candadito.

Reglas de la configuración de Nginx que no hay que cambiar

- `X-Real-IP $remote_addr`: la app la usa para limitar los intentos de login y los envíos del formulario. Sin esto, todos los visitantes cuentan como la misma persona.
- **No agregar** `X-Forwarded-Proto`: con esa cabecera, Next.js 16 responde **error 500** en las páginas que no existen (en vez de 404).
- `client_max_body_size 12m`: las fotos se suben de a una, hasta 10 MB.
- **Nada de** `proxy_cache`: la caché la maneja la app, y el admin la actualiza al guardar.
- `/uploads/` se sirve directo desde el disco (más rápido). Solo existen archivos `.webp`; cualquier otra cosa da 404.

---

11. Revisar la prueba

Recorré `https://nuevo.moramachadoconstrucciones.com` con esta lista:

- [ ] Inicio, Servicios, Obras, Nosotros, Contacto y Blog cargan, con fotos.
- [ ] Una obra inexistente (`/obras/no-existe`) muestra la página de error 404.
- [ ] `/admin` → iniciar sesión con el usuario del paso 8.1.
- [ ] En el admin: editar un texto en **Secciones** y ver el cambio en el sitio.
- [ ] Subir una foto en **Medios**.
- [ ] Enviar una consulta desde **Contacto** → aparece en **Consultas**. Hasta hacer el paso 13, el aviso por email figura como fallido: es lo esperado.
- [ ] `https://nuevo.moramachadoconstrucciones.com/robots.txt` dice `Disallow: /` (la prueba no se indexa).
- [ ] Desde el celular, con datos móviles.

Cuando todo esté bien, el cliente puede cargar el contenido real en este mismo admin: **la base de la prueba es la misma que se usa al lanzar**, no hay que cargar nada dos veces.

---

12. Emails: casillas en Webempresa

Las casillas info@, sofia@ y rodo@ están en **Webempresa**. Para que les lleguen emails, el DNS (que está en Vercel) tiene que decir "los emails de este dominio van a Webempresa". **Hoy ese registro no existe**, así que conviene resolverlo cuanto antes, aunque el sitio siga en Vercel: no depende del VPS.

12.1 Averiguar qué registros pide Webempresa

En el **cPanel de Webempresa**, buscá alguna de estas secciones (el nombre varía según la versión):

- **"Email Deliverability" / "Entregabilidad del correo"**: muestra los registros **MX**, **SPF** y **DKIM** que hacen falta, con los valores exactos.
- **"Zone Editor" / "Editor de zona"**: muestra la zona DNS que Webempresa tendría para el dominio.

Anotá:

Registro	Nombre	Tipo	Valor (de ejemplo, usá los de tu panel)
Correo entrante	`@` (el dominio)	`MX`	servidor de correo de Webempresa, con su prioridad
Servidor de correo	`mail` (si el MX apunta a `mail.moramachadoconstrucciones.com`)	`A`	IP del servidor de Webempresa
SPF	`@`	`TXT`	`v=spf1 … ~all` (lo que indique el panel)

Registro	Nombre	Tipo	Valor (de ejemplo, usá los de tu panel)
DKIM	`default._domainkey` (o el que indique)	`TXT`	`v=DKIM1; k=rsa; p=…`

> Si no lo encontrás, pedíselo al soporte de Webempresa: "*Tengo el DNS del dominio en otro proveedor. ¿Qué registros MX, SPF y DKIM tengo que crear para que funcionen las casillas?*"

12.2 Cargarlos en Vercel

Vercel → **Domains** → `moramachadoconstrucciones.com` → **DNS Records** → **Add Record**, uno por cada fila de la tabla.

- En Vercel, el nombre `@` (raíz) se carga dejando **Name vacío**.
- Para MX hay un campo **Priority**: poné la que indique Webempresa.

✅ **Comprobá [Mac]:**

```
dig +short MX moramachadoconstrucciones.com     # ahora aparece el servidor de Webempresa
dig +short TXT moramachadoconstrucciones.com    # aparece el v=spf1 …
```

Después mandá un email desde tu Gmail a `info@moramachadoconstrucciones.com` y revisá que llegue al webmail de Webempresa.

> 💡 **Regla de oro:** un dominio solo puede tener **un** registro SPF (`v=spf1`) en la raíz. El de Resend (paso 13) va en el subdominio `send` , así que no choca con el de Webempresa.

---

13. Emails del sitio: Resend

El sitio **envía** emails (el aviso de cada consulta a info@ y el email de prueba) a través de **Resend**. Resend **no tiene casillas**: solo envía. Las casillas siguen en Webempresa.

13.1 Dar de alta el dominio

1. Entrá a resend.com → **Domains** → **Add Domain** → `moramachadoconstrucciones.com` .
2. Elegí una región (la que sugiera; no se cambia después).
3. Resend te muestra **3 registros**. Tienen esta forma (**copiá los valores exactos de tu panel**):

Tipo	Name (en Vercel)	Valor	Prioridad	Para qué
`TXT`	`resend._domainkey`	`p=MIGfMA0GCSq…` (largo)	—	**DKIM**: firma los emails como del dominio

Tipo	Name (en Vercel)	Valor	Prioridad	Para qué
`MX`	`send`	`feedback-smtp.<región>.amazonses.com`	`10`	Avisos de rebote del subdominio de envío
`TXT`	`send`	`v=spf1 include:amazonses.com ~all`	—	**SPF** del subdominio de envío

4. Cargalos en **Vercel** → **DNS Records** igual que en el paso 12.2.
5. Volvé a Resend y tocá **Verify**. Puede tardar de minutos a unas horas en quedar **Verified**.

⚠️ No tocar los registros de la raíz

- Los registros de Resend van en `send` y en `resend._domainkey` (subdominios). **No** modifican el **MX de la raíz**, que es el de Webempresa (paso 12).
- **No** agregues `include:amazonses.com` al SPF de la raíz.
- Si algún panel ofrece "reemplazar los registros MX existentes", **no** aceptes.

13.2 DMARC (recomendado)

DMARC le dice a Gmail, Outlook, etc. qué hacer con los emails que dicen ser del dominio y no lo son. Empezamos solo **monitoreando** (`p=none` , no bloquea nada):

Tipo	Name	Valor
`TXT`	`_dmarc`	`v=DMARC1; p=none; rua=mailto:info@moramachadoconstrucciones.com`

Vas a recibir en info@ reportes periódicos (son archivos `.xml` , se pueden ignorar). Si después de unas semanas todo funciona bien, se puede subir a `p=quarantine` .

13.3 Crear la API key

1. Resend → **API Keys** → **Create API Key**.
2. **Name:** `VPS Mora` .
3. **Permission: Sending access** (no "Full access").
4. **Domain:** `moramachadoconstrucciones.com` .
5. Copiala: **se muestra una sola vez**. Guardala en tu gestor de contraseñas.

13.4 Cargarla en el VPS

[VPS]

```
nano /var/www/mora/shared/.env        # completá RESEND_API_KEY="re_…"
pm2 reload /var/www/mora/current/deploy/ecosystem.config.cjs --update-env
```

> Esta variable se lee al arrancar la app: alcanza con recargar, no hace falta otro deploy.

13.5 Probar

1. Admin → **Configuración** → sección **Formulario de contacto** → **Enviar email de prueba**.
2. Tiene que llegar a info@ (revisá también spam la primera vez).
3. Mandá una consulta desde **Contacto**. Llega a info@ con el asunto «Nueva consulta web – {nombre} – {teléfono}». Si respondés ese email, le escribís directamente a quien consultó.

> Si un envío falla, la consulta **se guarda igual** y en su detalle aparece el botón **Reintentar envío** con el error. En Resend → **Emails** se ve el historial de envíos.

---

14. Backups a Google Drive

Todas las noches a las 03:15, `deploy/backup.sh` guarda:

- la **base de datos** (`db-AAAA-MM-DD.dump`), y
- las **fotos subidas** (`uploads-AAAA-MM-DD.tar.gz`)

en `/var/www/mora/backups` (se borran a los 14 días). Pero si el VPS se rompe o se pierde, esas copias se pierden con él. Por eso las copiamos también a **tu Google Drive** con `rclone`.

14.1 Instalar rclone en tu Mac

El VPS no tiene navegador, así que el permiso de Google se da desde tu Mac. **[Mac]**:

```
brew install rclone
```

14.2 Configurar el remoto en el VPS

[VPS]

```
rclone config
```

Respondé así (los números de las opciones pueden cambiar según la versión: guiate por el **texto**):

Pregunta	Respuesta
`n) New remote`	`n`
`name>`	`gdrive`

Pregunta	Respuesta
`Storage>`	escribí `drive` (Google Drive)
`client_id>` / `client_secret>`	Enter (vacío)
`scope>`	la opción `drive.file` ("Access to files created by rclone only"): rclone solo ve lo que él mismo sube, no el resto de tu Drive
`service_account_file>`	Enter (vacío)
`Edit advanced config?`	`n`
`Use web browser to automatically authenticate?`	`n` (el VPS no tiene navegador)

Ahí rclone te muestra un comando del estilo:

```
rclone authorize "drive" "eyJzY29wZSI6…"
```

[Mac]: copiá ese comando completo y ejecutalo en tu Mac. Se abre el navegador: iniciá sesión con tu cuenta de Google y aceptá. La terminal de la Mac imprime un bloque `{"access_token":…}` (el **token**).

[VPS]: pegá ese token donde dice `config_token>` y seguí:

Pregunta	Respuesta
`Configure this as a Shared Drive?`	`n`
`Keep this "gdrive" remote?`	`y`
(menú principal)	`q` para salir

14.3 Crear la carpeta y activar la copia

[VPS]

```
rclone mkdir gdrive:mora-backups
rclone lsd gdrive:                          # tiene que aparecer "mora-backups"
nano /var/www/mora/shared/.env               # BACKUP_RCLONE_REMOTE="gdrive:mora-backups"
```

14.4 Probar un backup ahora

```
/var/www/mora/current/deploy/backup.sh
ls -lh /var/www/mora/backups
rclone ls gdrive:mora-backups
```

✅ **Comprobá:** en tu Google Drive aparece la carpeta **mora-backups** con los dos archivos de hoy.

La copia en Drive guarda **90 días** (`BACKUP_REMOTE_KEEP_DAYS`). Cada día ocupa más o menos lo que pesan las fotos subidas más un poco de base: revisá de vez en cuando el espacio libre de tu Drive (15 GB gratis).

14.5 Restaurar (por si alguna vez hace falta)

> ⚠️ Esto **reemplaza** los datos actuales por los del backup. Hacelo con calma y, si podés, probalo antes en una base aparte (ver abajo).

[VPS]

```
# 1) Si el backup está solo en Drive, bajalo:
rclone copy gdrive:mora-backups/db-AAAA-MM-DD.dump /var/www/mora/backups/
rclone copy gdrive:mora-backups/uploads-AAAA-MM-DD.tar.gz /var/www/mora/backups/

# 2) Parar la app
pm2 stop mora

# 3) Restaurar la base
pg_restore --clean --if-exists --no-owner \
  -d "postgresql://mora:<CONTRASEÑA_DB>@localhost:5432/mora_prod" \
  /var/www/mora/backups/db-AAAA-MM-DD.dump

# 4) Restaurar las fotos
tar -xzf /var/www/mora/backups/uploads-AAAA-MM-DD.tar.gz -C /var/www/mora

# 5) Volver a arrancar
pm2 start mora
```

Probar un backup sin tocar nada (recomendado una vez cada tanto):

```
sudo -u postgres createdb -O mora mora_restore_test
pg_restore --no-owner -d "postgresql://mora:
<CONTRASEÑA_DB>@localhost:5432/mora_restore_test" \
  /var/www/mora/backups/db-AAAA-MM-DD.dump
psql "postgresql://mora:<CONTRASEÑA_DB>@localhost:5432/mora_restore_test" -c 'select
count(*) from "Media";'
sudo -u postgres dropdb mora_restore_test
```

---

15. Monitoreo

Para enterarte si el sitio se cae **antes que el cliente**:

1. Creá una cuenta gratis en UptimeRobot (o Better Stack).
2. **New monitor** → **HTTP(s)** → URL: `https://nuevo.moramachadoconstrucciones.com/healthz` (después del lanzamiento: el dominio principal).
3. Intervalo: 5 minutos. Alerta: tu email.

`/healthz` responde `{"ok":true}` si la app y la base funcionan, y un error 503 si la base no responde.

---

16. Lanzamiento (con el OK del cliente)

Hacelo en un horario tranquilo (por ejemplo, un martes a la mañana) y con tiempo para revisar.

16.1 Un día antes

- **Vercel → DNS Records:** bajá el **TTL** de los registros del dominio a `60` (si se puede editar). Así el cambio se nota en minutos y no en horas.
- Confirmá que el paso 12 (casillas) y el 13 (Resend) funcionan.
- Hacé un backup a mano: `/var/www/mora/current/deploy/backup.sh`.

16.2 Pasar el código a `main`

[Mac]

```
git checkout main
git merge develop
git push origin main
git checkout develop
```

> `main` hoy tiene el sitio viejo (el HTML estático). Después del merge, `main` es la web nueva. El sitio viejo sigue en el historial de git (commit `39b8b90`).

16.3 Cambiar el `.env` a producción

[VPS]

```
nano /var/www/mora/shared/.env
```

```
SITE_URL="https://moramachadoconstrucciones.com"
SITE_NOINDEX="false"
```

16.4 Apuntar el dominio al VPS

En **Vercel**:

1. **Proyecto del sitio viejo** → **Settings** → **Domains** → quitá `moramachadoconstrucciones.com` y `www.moramachadoconstrucciones.com` **del proyecto**.

   > ⚠️ Quitalos del **proyecto**, **no** del **equipo/cuenta** (Vercel → Domains). Si borrás el dominio de la cuenta, **se borra toda la zona DNS**, incluidos los registros de Webempresa y Resend.

2. **Domains** → `moramachadoconstrucciones.com` → **DNS Records**: borrá los registros `A` / `ALIAS` / `CNAME` de la raíz y de `www` que apunten a Vercel (si quedó alguno) y creá:

Name	Type	Value
(vacío = raíz)	`A`	`201.32.131.60`
`www`	`A`	`201.32.131.60`

No toques los MX ni los TXT (son de los emails).

✅ **Comprobá [Mac]:**

```
dig +short moramachadoconstrucciones.com      # 201.32.131.60
dig +short www.moramachadoconstrucciones.com  # 201.32.131.60
dig +short MX moramachadoconstrucciones.com   # sigue el de Webempresa
```

16.5 Nginx y certificado del dominio principal

[VPS], cuando los `dig` ya devuelvan `201.32.131.60`:

```
# http del dominio principal
sudo cp /var/www/mora/current/deploy/nginx/mora-http.conf /etc/nginx/sites-available/mora-
http
sudo ln -s /etc/nginx/sites-available/mora-http /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx

# certificado (dominio + www)
sudo certbot certonly --webroot -w /var/www/certbot \
  -d moramachadoconstrucciones.com -d www.moramachadoconstrucciones.com \
  --deploy-hook "systemctl reload nginx"

# https del dominio principal
sudo cp /var/www/mora/current/deploy/nginx/mora.conf /etc/nginx/sites-available/mora
sudo ln -s /etc/nginx/sites-available/mora /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

16.6 Deploy de `main`

```
/var/www/mora/current/deploy/deploy.sh main
```

Es necesario porque `SITE_URL` y `SITE_NOINDEX` cambiaron, y se leen al buildear.

16.7 Revisar

- [ ] `https://moramachadoconstrucciones.com` carga la web nueva con candadito.
- [ ] `http://moramachadoconstrucciones.com` y `https://www.moramachadoconstrucciones.com` redirigen a `https://moramachadoconstrucciones.com`.
- [ ] `/robots.txt` **ya no** dice `Disallow: /` (ahora muestra `Disallow: /admin` y el sitemap).
- [ ] Una consulta de prueba llega a info@.
- [ ] El monitor de UptimeRobot apunta al dominio principal.
- [ ] **Google Search Console:** agregá la propiedad del dominio y enviá `https://moramachadoconstrucciones.com/sitemap.xml`.

16.8 Después del lanzamiento

- **Subdominio de prueba:** podés dejarlo o quitarlo:

  ```
  sudo rm /etc/nginx/sites-enabled/mora-staging /etc/nginx/sites-enabled/mora-staging-
  http
  sudo nginx -t && sudo systemctl reload nginx
  sudo certbot delete --cert-name nuevo.moramachadoconstrucciones.com
  ```

  y borrá el registro `nuevo` en Vercel.

- **Repo privado:** GitHub → repo → **Settings** → **Danger Zone** → **Change visibility** → Private. El VPS ya usa la deploy key (paso 7), así que no se rompe nada.
- **Proyecto viejo de Vercel:** se puede borrar el **proyecto** (no el dominio de la cuenta: ahí vive el DNS). En el repo local, borrá la carpeta `.vercel/`.

> **¿Sacar el DNS de Vercel?** Es opcional. El DNS de Vercel es gratis y funciona aunque no alojes nada ahí. Si algún día lo querés mover (a GoDaddy, Webempresa o Cloudflare), primero recreá **todos** los registros en el proveedor nuevo y recién después cambiá los nameservers en GoDaddy.

---

17. El día a día

Publicar una actualización del código

[Mac]: commit + `git push` a la rama correspondiente (`develop` en la prueba, `main` después del lanzamiento). **[VPS]**:

```
/var/www/mora/current/deploy/deploy.sh main       # o develop
```

El sitio sigue funcionando durante el build; el corte es de 1-2 segundos al final, cuando PM2 reinicia.

> **Cambios en la base (migraciones):** se aplican **antes** del build, mientras la versión anterior sigue atendiendo. Por eso tienen que ser "aditivas": agregar columnas o tablas, no borrarlas ni renombrarlas en el mismo deploy.

Volver a la versión anterior

```
ls -1t /var/www/mora/releases           # la primera es la actual, la segunda la anterior
ln -sfn /var/www/mora/releases/<CARPETA_ANTERIOR> /var/www/mora/current
pm2 reload /var/www/mora/current/deploy/ecosystem.config.cjs --update-env
```

> Esto vuelve atrás el **código**, no la base de datos.

Ver qué está pasando

Quiero ver…	Comando
Si la app está andando	`pm2 status`
Los logs de la app en vivo	`pm2 logs mora` (salir con `Ctrl+C`)

Quiero ver…	Comando
Errores de la app	`tail -n 100 /var/www/mora/logs/app.err.log`
Errores de Nginx	`sudo tail -n 100 /var/log/nginx/mora.error.log`
El último backup	`tail -n 5 /var/www/mora/logs/backup.log`
Espacio en disco	`df -h /`
Memoria	`free -h`
Certificados y vencimiento	`sudo certbot certificates`

Tareas mensuales (10 minutos)

- [ ] `sudo apt update && sudo apt upgrade -y` (si pide reiniciar: `sudo reboot`; PM2 levanta la app solo).
- [ ] `df -h /`: que el disco no pase del 80 %.
- [ ] Mirar que en Google Drive estén los backups de los últimos días.
- [ ] `sudo certbot renew --dry-run`: simula la renovación del certificado.

Reiniciar la app a mano

```
pm2 reload mora
```

---

18. Si algo falla

Síntoma	Causa probable	Qué hacer
502 Bad Gateway	La app no está corriendo.	`pm2 status`; si no está "online": `pm2 logs mora --lines 50` para ver el error, y `pm2 reload mora`.
El deploy se corta en **Build** con `Killed`	Falta memoria.	Agregá swap (paso 3.4) y volvé a correr el deploy.
El deploy termina con `✗ /healthz no responde`	La versión nueva no arranca (el script ya volvió a la anterior).	`pm2 logs mora --lines 100`. Suele ser una variable faltante o mal escrita en `.env`.
`Error: P1001: Can't reach database`	Postgres apagado o `DATABASE_URL` mal.	`sudo systemctl status postgresql`; probá la conexión del paso 5.2.
`Permission denied (publickey)` al hacer `git fetch`	La deploy key no está cargada en GitHub.	Repetí el paso 7 y probá `ssh -T git@github.com`.

Síntoma	Causa probable	Qué hacer
certbot falla con **challenge** / **unauthorized**	El DNS todavía no apunta al VPS, o el puerto 80 está cerrado.	`dig +short <dominio>` tiene que dar la IP del VPS; `sudo ufw status`.
Las fotos nuevas no se ven (404)	Permisos de la carpeta.	`ls -ld /var/www/mora/uploads` tiene que mostrar `drwxr-xr-x`; `chmod 755 /var/www/mora /var/www/mora/uploads`.
Los emails de consultas no llegan	Falta la API key o el dominio no está verificado en Resend.	Abrí la consulta en el admin: muestra el error. Revisá Resend → Domains y Emails.
Los emails llegan a spam	Faltan DKIM/SPF/DMARC o todavía no propagaron.	Revisá el paso 13 y que el dominio figure **Verified** en Resend.
A info@ no le llegan emails de nadie	Faltan los MX de Webempresa.	Paso 12.
Algunas páginas devuelven **500**	¿Alguien agregó `X-Forwarded-Proto` en Nginx?	Ver las reglas del paso 10.
El sitio muestra contenido viejo después de editar	Hay más de una instancia de la app, o se agregó caché en Nginx.	`pm2 status` tiene que mostrar **una** sola `mora`; nada de `proxy_cache` en Nginx.
Me quedé afuera del SSH (o `Too many authentication failures`)	Firewall o configuración de SSH.	Entrá por la **consola web** del panel de DonWeb y revisá `sudo ufw status` y `/etc/ssh/sshd_config.d/`.

Anexo: cómo funciona por dentro



Archivo	Para qué
`deploy.sh`	Deploy con release nueva, migraciones, seed idempotente, build, cambio de versión y vuelta atrás automática si `/healthz` falla. Deja las 3 últimas releases.
`ecosystem.config.cjs`	Proceso PM2 `mora`. **Una sola instancia** a propósito: la caché del sitio vive dentro del proceso, y dos instancias mostrarían contenido desactualizado.
`nginx/mora-http.conf`	Puerto 80: desafío de Let's Encrypt y redirección a https.

Archivo	Para qué
`nginx/mora.conf`	Sitio https: `www` → dominio, fotos desde disco, proxy a la app, compresión.
`backup.sh`	Volcado de la base + fotos, retención local (14 días) y en Drive (90 días).
`crontab`	Backup a las 03:15 y limpieza de sesiones vencidas a las 03:45.

Detalles técnicos:

- **Releases como `git worktree`:** cada deploy es una carpeta nueva conectada al repo. Así el seed puede leer las fotos del sitio viejo del historial de git.
- **El build lee la base** (Next.js genera las páginas por adelantado con el contenido real). Por eso el seed corre antes del build: en el primer deploy la base empieza vacía.
- **404 reales:** las páginas de obras, servicios y notas se chequean en `src/proxy.ts` antes de renderizar. Si el slug no existe o no está publicado, responde 404 (sin eso, Next.js respondería 200 con la página de error).
- **Variables leídas al buildear:** `SITE_URL` y `SITE_NOINDEX`. El resto se lee al arrancar la app (alcanza con `pm2 reload … --update-env`).

PARTE 3 — Archivos de `deploy/` y configuración (copias exactas)

`deploy/deploy.sh`

```
#!/usr/bin/env bash
# Builds a new release from git and switches to it with PM2, rolling back if
# the health check fails. Run on the server as the app user:
#   /var/www/mora/current/deploy/deploy.sh [branch]      (default: main)
# First deploy: see deploy/README.md (the script lives in the repo clone too).
set -Eeuo pipefail

APP_ROOT="${APP_ROOT:-/var/www/mora}"
BRANCH="${1:-main}"
KEEP="${KEEP_RELEASES:-3}"
PORT="${PORT:-3000}"
# Corepack would otherwise ask "download pnpm? [Y/n]" and block the script.
export COREPACK_ENABLE_DOWNLOAD_PROMPT=0

log() { printf '\n\033[1m▶ %s\033[0m\n' "$*"; }

cd "$APP_ROOT/repo"
log "Actualizando repo ($BRANCH)"
git fetch --prune origin
SHA="$(git rev-parse "origin/$BRANCH")"
REL="$APP_ROOT/releases/$(date +%Y%m%d-%H%M%S)-${SHA:0:7}"
PREV="$(readlink -f "$APP_ROOT/current" 2>/dev/null || true)"

# A worktree (not an export) keeps git history available: the seed reads the
# legacy images from commit 39b8b90.
git worktree add --detach "$REL" "$SHA"
cd "$REL"
ln -s "$APP_ROOT/shared/.env" .env
set -a; . ./.env; set +a

log "Instalando dependencias"
pnpm install --frozen-lockfile

log "Migraciones"
# Before the build: prerendering reads the database with the new schema.
# Keep migrations additive so the running release keeps working meanwhile.
pnpm prisma migrate deploy

log "Contenido base"
# Idempotent: full seed only on an empty database; otherwise it just adds
# missing content blocks/fields and the brand assets (never overwrites edits).
pnpm db:seed
```

```
log "Build"
pnpm build
cp -r public .next/standalone/
cp -r .next/static .next/standalone/.next/

log "Activando release ${SHA:0:7}"
ln -sfn "$REL" "$APP_ROOT/current"
pm2 startOrReload "$REL/deploy/ecosystem.config.cjs" --update-env

log "Chequeo de salud"
ok=0
for _ in $(seq 1 30); do
  if curl -fsS "http://127.0.0.1:$PORT/healthz" >/dev/null; then ok=1; break; fi
  sleep 2
done
if [ "$ok" -ne 1 ]; then
  echo "✗ /healthz no responde."
  if [ -n "$PREV" ] && [ -d "$PREV" ]; then
    echo "  Volviendo a $(basename "$PREV")"
    ln -sfn "$PREV" "$APP_ROOT/current"
    pm2 startOrReload "$PREV/deploy/ecosystem.config.cjs" --update-env
  fi
  exit 1
fi
pm2 save >/dev/null

log "Limpiando releases viejas (quedan $KEEP)"
cd "$APP_ROOT/repo"
ls -1dt "$APP_ROOT"/releases/*/ | tail -n +"$((KEEP + 1))" | while read -r old; do
  [ "$(readlink -f "$APP_ROOT/current")/" = "$old" ] && continue
  git worktree remove --force "$old"
done
git worktree prune

echo "✓ Deploy ${SHA:0:7} listo."
```

`deploy/ecosystem.config.cjs`

```
// PM2 process for the Next.js standalone server.
// One instance in fork mode on purpose: Next's cache ('use cache' + updateTag)
// lives in this process, so a second instance would serve stale content.
const ROOT = process.env.APP_ROOT || "/var/www/mora";

module.exports = {
  apps: [
    {
      name: "mora",
      // `current` is a symlink to the active release (see deploy.sh).
      cwd: `${ROOT}/current/.next/standalone`,
      script: "server.js",
      node_args: `--env-file=${ROOT}/shared/.env`,
      exec_mode: "fork",
      instances: 1,
      env: { NODE_ENV: "production", PORT: "3000", HOSTNAME: "127.0.0.1" },
      max_memory_restart: "800M",
      kill_timeout: 10000,
      time: true,
      out_file: `${ROOT}/logs/app.out.log`,
      error_file: `${ROOT}/logs/app.err.log`,
    },
  ],
};
```

`deploy/nginx/mora-http.conf`

```
# Port 80: Let's Encrypt challenges + redirect to https.
# Install: sudo cp deploy/nginx/mora-http.conf /etc/nginx/sites-available/mora-http
#          sudo ln -s ../sites-available/mora-http /etc/nginx/sites-enabled/
server {
    listen 80;
    listen [::]:80;
    server_name moramachadoconstrucciones.com www.moramachadoconstrucciones.com;

    location ^~ /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }

    location / {
        return 301 https://moramachadoconstrucciones.com$request_uri;
    }
}
```

`deploy/nginx/mora.conf`

```
# HTTPS site. Enable only after the certificate exists (see deploy/README.md).
# For a staging subdomain, copy with:  sed
's/moramachadoconstrucciones\.com/nuevo.moramachadoconstrucciones.com/g'
# (and remove the www server block).

upstream mora_app {
    server 127.0.0.1:3000;
    keepalive 16;
}

# www → apex
server {
    listen 443 ssl;
    listen [::]:443 ssl;
    http2 on;
    server_name www.moramachadoconstrucciones.com;

    ssl_certificate     /etc/letsencrypt/live/moramachadoconstrucciones.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/moramachadoconstrucciones.com/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;

    return 301 https://moramachadoconstrucciones.com$request_uri;
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;
    http2 on;
    server_name moramachadoconstrucciones.com;

    ssl_certificate     /etc/letsencrypt/live/moramachadoconstrucciones.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/moramachadoconstrucciones.com/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;

    # Media uploads go one file per request (max 10 MB, see lib/media/process.ts).
    client_max_body_size 12m;
    server_tokens off;

    gzip on;
    gzip_vary on;
    gzip_proxied any;
    gzip_types text/plain text/css application/javascript application/json application/xml
```

```
image/svg+xml text/x-component;

    access_log /var/log/nginx/mora.access.log;
    error_log  /var/log/nginx/mora.error.log warn;

    # Uploaded media straight from disk (UPLOAD_DIR). Only .webp variants exist;
    # anything else under /uploads/ is a 404. These responses skip Next, so the
    # security headers that matter for files are set here.
    location ~ ^/uploads/(.+\.webp)$ {
        alias /var/www/mora/uploads/$1;
        access_log off;
        add_header Cache-Control "public, max-age=31536000, immutable" always;
        add_header X-Content-Type-Options "nosniff" always;
        add_header Cross-Origin-Resource-Policy "same-site" always;
    }
    location /uploads/ {
        return 404;
    }

    location / {
        proxy_pass http://mora_app;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        # The app trusts X-Real-IP for rate limiting: always overwrite it here.
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        # No X-Forwarded-Proto on purpose: with "https" Next 16 treats the
        # proxy.ts 404 rewrite as an external https fetch to itself and fails
        # (500). Absolute URLs come from SITE_URL, so the app doesn't need it.
        proxy_set_header X-Forwarded-Host $host;
        proxy_read_timeout 60s;
        proxy_send_timeout 60s;
    }
}
```

`deploy/backup.sh`

```
#!/usr/bin/env bash
# Daily backup: database (pg_dump custom format) + uploaded media.
# Keeps BACKUP_KEEP_DAYS days locally; copies off-site when BACKUP_RCLONE_REMOTE
# is set in shared/.env (e.g. "gdrive:mora-backups", configured with `rclone config`).
set -Eeuo pipefail

APP_ROOT="${APP_ROOT:-/var/www/mora}"
DEST="$APP_ROOT/backups"
set -a; . "$APP_ROOT/shared/.env"; set +a
KEEP_DAYS="${BACKUP_KEEP_DAYS:-14}"
STAMP="$(date +%F)"

mkdir -p "$DEST"
# pg_dump rejects Prisma's ?schema=public query parameter.
pg_dump --format=custom --no-owner --file="$DEST/db-$STAMP.dump" "${DATABASE_URL%%\?*}"
tar -czf "$DEST/uploads-$STAMP.tar.gz" -C "$(dirname "$UPLOAD_DIR")" "$(basename
"$UPLOAD_DIR")"

find "$DEST" -type f -mtime +"$KEEP_DAYS" -delete

if [ -n "${BACKUP_RCLONE_REMOTE:-}" ]; then
  rclone copy "$DEST" "$BACKUP_RCLONE_REMOTE" --max-age 48h
  # Off-site retention (default 90 days) so the Drive doesn't fill up.
  rclone delete "$BACKUP_RCLONE_REMOTE" --min-age "${BACKUP_REMOTE_KEEP_DAYS:-90}d"
fi
echo "[backup] $(date "+%F %T") ok: $(du -sh "$DEST" | cut -f1) en $DEST"
```

`deploy/crontab`

```
# Install for the app user with:  crontab deploy/crontab   (merge if it already has
entries)
SHELL=/bin/bash
PATH=/usr/local/bin:/usr/bin:/bin
APP_ROOT=/var/www/mora
COREPACK_ENABLE_DOWNLOAD_PROMPT=0

# 03:15 backup of database + uploads
15 3 * * * $APP_ROOT/current/deploy/backup.sh >> $APP_ROOT/logs/backup.log 2>&1
# 03:45 remove expired sessions and rate-limit windows
45 3 * * * cd $APP_ROOT/current && pnpm -s db:cleanup >> $APP_ROOT/logs/cleanup.log 2>&1
```

`.env.example`

```
# ── Database ──────────────────────────────────────────────
# PostgreSQL 16. Dev uses Homebrew postgresql@16 on port 5433.
DATABASE_URL="postgresql://USER:PASSWORD@localhost:5432/DB?schema=public"

# ── Site ──────────────────────────────────────────────────
# Public base URL, no trailing slash (canonical, sitemap, OG, email links).
SITE_URL="https://moramachadoconstrucciones.com"

# "true" on a staging host (e.g. nuevo.<dominio>) → robots.txt disallows everything.
# Read at build time: rebuild after changing it.
SITE_NOINDEX="false"

# Absolute path where uploaded media is stored (outside the repo in prod).
# Nginx serves it at /uploads/. In dev a route handler serves it.
UPLOAD_DIR="/var/www/mora/uploads"

# ── Auth (F6) ─────────────────────────────────────────────
# 32+ random bytes: `openssl rand -base64 32`
AUTH_SECRET=""
# Salt used to hash lead IPs (privacy): `openssl rand -base64 16`
IP_HASH_SALT=""

# ── Email (Resend) ────────────────────────────────────────
# API key with "Sending access" restricted to the domain (see deploy/README.md).
# Empty in dev → emails are printed to the console instead of being sent.
RESEND_API_KEY=""
MAIL_FROM="Web Mora Machado <no-reply@moramachadoconstrucciones.com>"
# Fallback recipient when Settings → "Emails que reciben las consultas" is empty.
MAIL_TO_DEFAULT="info@moramachadoconstrucciones.com"
# "true" → also send an acknowledgement to the person who wrote (if they left an email).
LEAD_CONFIRMATION_EMAIL="false"

# ── Optional: Cloudflare Turnstile on the contact form ────
TURNSTILE_SITE_KEY=""
TURNSTILE_SECRET_KEY=""

# ── Backups (deploy/backup.sh) ────────────────────────────
BACKUP_KEEP_DAYS="14"
# Off-site copy with rclone, e.g. "gdrive:mora-backups" (empty = local only).
BACKUP_RCLONE_REMOTE=""
# Days kept in the off-site copy.
BACKUP_REMOTE_KEEP_DAYS="90"
```

```
# ── Local dev only ──────────────────────────────────────────
# Test admin created with `pnpm create-admin`; Playwright e2e logs in with it.
DEV_ADMIN_EMAIL="admin@mora.test"
DEV_ADMIN_PASSWORD=""
```

`src/proxy.ts`

```
import { NextResponse, type NextRequest } from "next/server";
import { SESSION_COOKIE, verifySessionToken } from "@/lib/auth/token";
import { publicSlugExists, type DetailSection } from "@/lib/content/exists";

const DETAIL_RE = /^\/(servicios|obras|blog)\/([^/]+)\/?$/;
/** Draft Mode cookie (admin preview of unpublished items). */
const DRAFT_COOKIE = "__prerender_bypass";

export async function proxy(request: NextRequest) {
  const { pathname } = request.nextUrl;
  const detail = DETAIL_RE.exec(pathname);
  if (detail) return detailGuard(request, detail[1] as DetailSection, detail[2]!);
  return adminGuard(request);
}

/**
 * Detail pages stream, so once rendering starts the status is already 200 and
 * notFound() can only produce a soft 404. Unknown/unpublished slugs are answered
 * here, before rendering, with a real 404 (rewrite to an unmatched path).
 */
async function detailGuard(request: NextRequest, section: DetailSection, slug: string) {
  if (request.cookies.has(DRAFT_COOKIE)) return NextResponse.next();
  let decoded = "";
  try {
    decoded = decodeURIComponent(slug);
  } catch {
    // malformed escape → not found
  }
  if (decoded && (await publicSlugExists(section, decoded))) return NextResponse.next();
  // Behind Nginx this only stays an internal rewrite if the request is plain
  // http: Nginx must NOT send X-Forwarded-Proto (see deploy/nginx/mora.conf).
  const url = request.nextUrl.clone();
  url.pathname = "/404";
  return NextResponse.rewrite(url);
}

/**
 * Optimistic auth check for /admin (JWT only, no DB). The real check —
 * session row + active user + role — runs in requireUser() on every page/action.
 */
async function adminGuard(request: NextRequest) {
  const { pathname, search } = request.nextUrl;
```

```
  const isLogin = pathname === "/admin/login";
  const token = request.cookies.get(SESSION_COOKIE)?.value;
  const claims = token ? await verifySessionToken(token) : null;

  if (!claims && !isLogin) {
    const url = new URL("/admin/login", request.url);
    url.searchParams.set("next", pathname + search);
    return NextResponse.redirect(url);
  }
  if (claims && isLogin) {
    // Token valid but session revoked/expired server-side → clear it and show login.
    if (request.nextUrl.searchParams.has("expired")) {
      const res = NextResponse.next();
      res.cookies.delete(SESSION_COOKIE);
      return res;
    }
    return NextResponse.redirect(new URL("/admin", request.url));
  }

  const res = NextResponse.next();
  res.headers.set("X-Robots-Tag", "noindex, nofollow");
  res.headers.set("Cache-Control", "private, no-store");
  return res;
}

export const config = {
  matcher: ["/admin", "/admin/:path*", "/servicios/:slug", "/obras/:slug", "/blog/:slug"],
};
```

`src/app/healthz/route.ts`

```
import { db } from "@/lib/db";

/** Liveness + DB check for uptime monitors and the deploy smoke test. Not cached. */
export async function GET() {
  try {
    await db.$queryRaw`SELECT 1`;
    return Response.json({ ok: true }, { headers: { "Cache-Control": "no-store" } });
  } catch {
    return Response.json({ ok: false }, { status: 503, headers: { "Cache-Control": "no-
store" } });
  }
}
```